UNIVERSITI PENDIDIKAN SULTAN IDRIS

اونيۏرسيتي ڤنديديقن سلطان ادريس

SULTAN IDRIS EDUCATION UNIVERSITY

Software Verification and Validation (DES3103)

Course Learning Outline

1. Briefing Session (Ice Breaking via Whiteboard, Class rules & Pro forma): **M. Teams**
2. Introduction to Software Verification & Validation I & II (V&V)
3. Managing V&V with SQA Integration
4. Quality metrics or attributes in V&V

WEEK 6-MID-TERM TEST (QUIZ): Topic 1 – 3-(CLO1:C2) – 20 MARKS

WEEK 7-PHASE 1: ASSIGNMENT 1-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO3:A5) - 20 MARKS: M. Teams

WEEK 9-PHASE 1: PRESENTATION & REFLECTION ON PHASE 1: M. Teams

5. Software Reviews – Will be covered on Assignments 1 & 2
6. Quality Control: Software Testing - Will be covered in Assignments 1 & 2
7. Human-Computer Interface Testing - Will be covered in Assignments 1 & 2
8. Manual Testing and Automation Testing: Will be covered in Final Assessment

WEEK 11-PHASE 2: ASSIGNMENT 2-ONLINE PROJECT-BASED LEARNING (OPBL)-(CLO2&4:P4&C5) - 20 MARKS: M. Teams
WEEK 13-PHASE 2: PRESENTATION & REFLECTION ON PHASE 2: M. Teams

9. Quality Maturity Models
10. Software Problem Analysis and Reporting
11. Quality Culture in doing V&V.
12. Safety and Security in V&V

WEEK 14: REFLECTION ON IMPLEMENTATION OPBL (SOFTWARE TESTERS) IN THE V&V COURSE: e-FORUM

WEEK 16-PHASE 3: FINAL ASSESSMENT – INDIVIDUAL SPECIAL PROJECT BASED ON PHASE 2 (CLO4:C5) (2 WEEKS) - 40 MARKS : e-ASSESSMENT

TOPIC 6

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Quality Control: Software Testing

- Overview of software testing as quality control
- Black box and white box testing
- Software testing implementation –

i. testing process
ii. test plan
iii. test case design Software
iv. test report (STR)
v. user acceptance test (UAT)

(according to IEEE recommendations)

Managing V&V with SQA Integrating SQA - Quality control in software development process

- The implementation of software quality control is a much more recent development.
- As the Information Age dawned in the mid-1900s, it became apparent that quality control standards should be applied to the development of software, just like the manufacturing of physical products.
- Though digital and physical products are quite dissimilar, the application of quality control standards shares a common goal to both: the production of products that consistently serve users as designed.

Differences Between Quality Control and Quality Assurance

The terms 'quality control' and 'quality assurance' are often used interchangeably, but they each encapsulate separate tasks and activities. While quality assurance focuses more on ensuring quality during the software development process, quality control is more about identifying defects and flaws in completed software products. Thus, quality assurance is more prevention-oriented, while quality control is more detection-oriented.

The scope of the two are also different. Quality assurance tends to be more organizational in breadth, focusing on all software development efforts that occur within an enterprise. Software quality control is comprised of a set of activities that focus upon monitoring the quality of a single software product — though likely duplicated across an organization's entire suite of software products.

And while quality assurance is used during most phases of the Software Development Life Cycle, quality control is typically restricted to the review and testing phases. However, there is **one common activity** that's integral to both quality assurance and quality control: **testing**.

Quality Control: Software Testing - Overview of software testing as quality control

Software tests are intellectual assets, too, and are as valuable as source code to a software project.
Over the long term, maintainable software tests significantly lower a project's cost.
It is very difficult, however, to write maintainable software tests, especially executable ones.
Existing approaches - including natural language, tables or forms, test scripts, programming languages, and test description languages - all are problematic, in Figure 1, evaluation of software test description practices.
Another solution is a test description language that provides a mechanism to specify software tests while separating different concerns of automated software testing.
Figure 1, analyze the current situation of the software test description. Here, propose a description language just for software testers: TestTalk. And present examples to illustrate the benefits of TestTalk and discuss implementation issues of this language.

Evaluation Criteria	Informal Languages	Scripting or Programming Languages	Test Description Languages	TestTalk
Writability/ Readability	High, but difficult to control quality	Low	High	***Very high***
Understandability	Potentially high, but ambiguous	Low	High	***Very high***
Test Input Data Provision/ Automation	Possible (depends on effort), more common in structured formats; Not automated	***Yes***, but mixed with other information; Automated	No, test data generation left to tester; Automation depends on tool support	***Yes***, clearly separated; Automated
Test Oracle Provision/ Automation	Possible (depends on effort), expected outputs common in structured formats; Not automated	***Yes***, but mixed with other information; Automated	No; Not automated	***Yes***, clearly separated; Automated
Maintainability	Potentially high, but depends on quality	Very Low	High	***Very high***
Portability	***Very high***	Low	***Very high***	***Very high***
Regression Testability	Low, and high rework required	Low, and high rework required	Medium, depends on tool support	***High*** (only transformation rules change
Manageability	Depends on quality	Very low	Possible, depends on tool support	***Very High***
Measurability	Depends on quality	Low	***High***, for black box testing	Possible (future work)

Figure 1. Evaluation of software test description practices

Quality Control: Software Testing - Black box and white box testing

Commercial off the shelf component (COTs)

- COTs is a software product that is developed for the general market, i.e. for the large customers, and that is delivered to many customers in identical format.

Black Box Testing	White Box Testing
1. Black box testing techniques are also called **functional** testing techniques	1. White box testing techniques are also called **structural** testing techniques.
2 Black Box Testing is a software testing method in which the internal structure/design/ implementation of the item being tested is **NOT known** to the tester	2. White Box Testing is a software testing method in which the internal structure design/implementation of the item being tested is **known** to the tester.
3. Mainly applicable to **lower levels** of testing such as Unit Testing and Integration Testing	3. It is mainly applicable to **higher levels** of testing such as Acceptance Testing and System Testing
4. Black box testing is generally done by **Software Testers**	4. White box testing is generally done by **Software Developers**
5. Programming knowledge is **NOT required**	5. Programming knowledge **is required**
6. Implementation knowledge is NOT required	6. Implementation knowledge **is required**
7. Basis for test cases & **requirement specification**	7. Basis for test cases & **detailed design**
8. Techniques that are used in Black Box testing are **Boundary-value analysis, Error guessing, Syntax testing, & Equivalence partitioning**	8. Techniques that are used in white box testing are **code coverage, data flow testing, & path testing**
9. **NOT suited** for algorithm testing	9. **Suitable** for algorithm testing
10. It is **least** time consuming and exhaustive	10. It is exhaustive and most time-consuming type of technique.
11. Best example of Black box testing is **Search on Google**. A user just enters keywords and gets the expected results in turn	11. Example: The tester chooses inputs to verify **loops, control flows, return types,** etc, through the code and determines the suitable output

Application of black box testing and white box testing to evaluate COTs components :

- The customer has no access to source code in COTS products. This forces testers to adopt an external, black-box, test approach.
- Although black-box testing is certainly not foreign to testers, it limits the view and expands the scope of testing. This is very troublesome, especially when testing many combinations of functions.
- Functional testing is redundant by its very nature. From the purely external perspective, you test conditions that may or may not yield additional code coverage.
- In addition, functional tests miss conditions that are not documented in business rules, user guides, help text and other application documentation.
- The bottom line is that in functional testing, you can test against a defined set of criteria, but there will likely be features and behaviour that the criteria will not include.
- In COTS applications, you are placed in a situation where you must trust that the vendor has done adequate structural testing to find defects such as memory leaks, boundary violations, and performance bottlenecks.
- Solution Strategies: Avoid complex combinations of tests and the idea of "testing everything." Instead, base tests on functional or business processes used in the real world environment.
- The initial tendency of people in testing COTS applications is to start defining tests based on user interfaces and all of the combinations of features

Quality Control: Software Testing - Black box and white box testing

Commercial off the shelf component (COTs)

VVT engineers must have deep understanding of system specifications as well as stakeholders' expectations. They must be capable of judiciously hypothesizing undesired system responses that have not been specified, even those that have not shown up in previously engineered systems. The very nature of black - box testing (i.e., not having to know the internal structure of the system) generally precludes test engineers from applying extra test efforts in verifying fragile elements of the system design. In fact, in black - box testing, test engineers are naturally oblivious to the internal workings of the unit being tested. The structure of this chapter and a proposed system testing taxonomy is depicted in Figure 2.

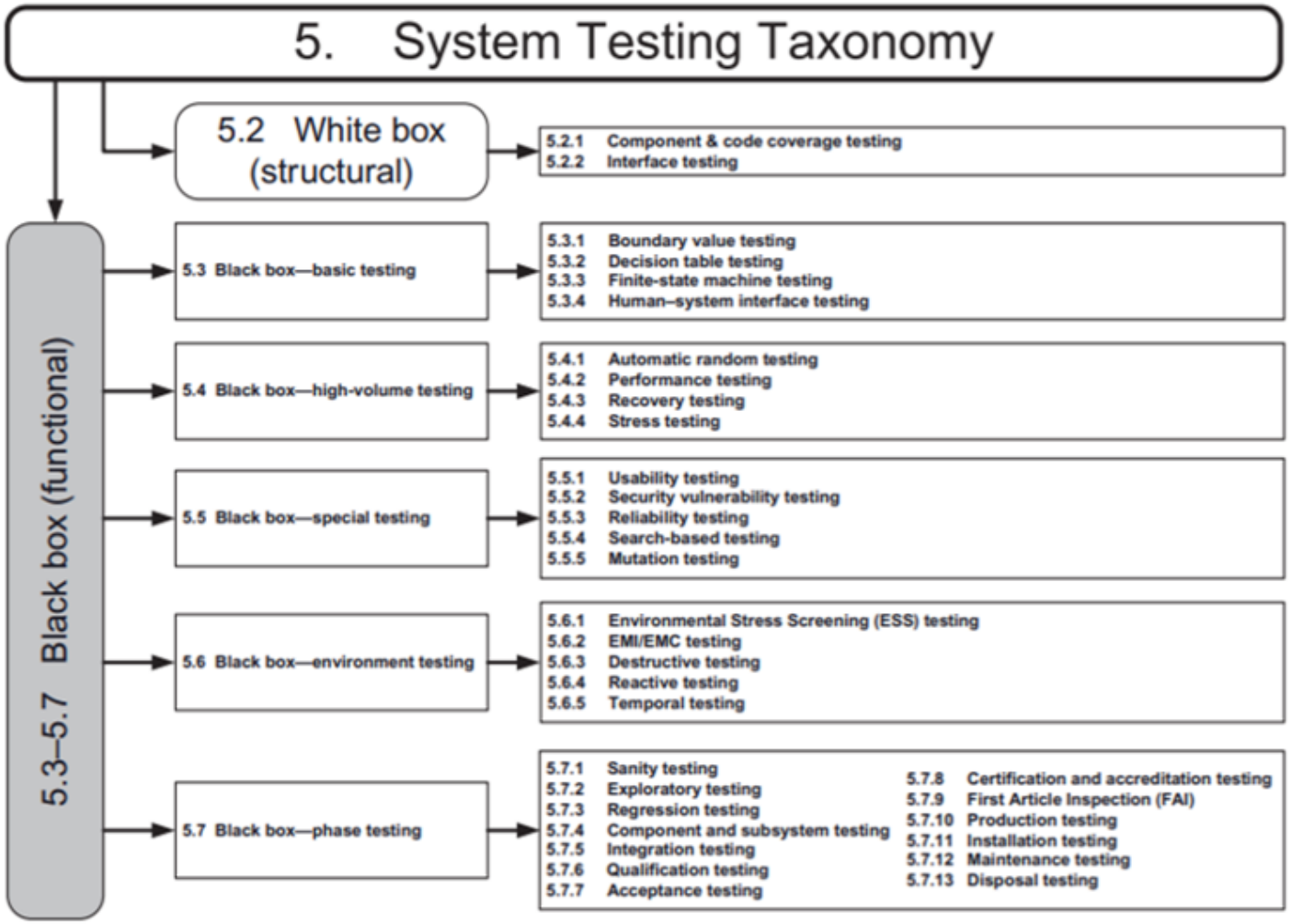


Figure 2. Chapter structure and system testing taxonomy

VERIFICATION, VALIDATION, AND TESTING OF ENGINEERED SYSTEMS - AVNER ENGEL Pg 377.

i. Software testing implementation - testing process, test plan, test case design Software and test report (STR) - *(according to IEEE recommendations)*

Testing Process

The purpose of the system Implementation phase is to create the elements of the system. Some elements may be purchased from other producers and therefore may require purchase specifications. Other elements may require detailed engineering design. Each element, whether purchased or built by the system producer, should be verified against its design and then tested to ensure its stand - alone compliance with its allocated requirements.

VVT activities during the system implementation phase include detailed planning of the testing process as well as performing simulation, analysis or actual testing, mostly at the subsystem level, in order to verify detailed designs/ specifications against requirements.

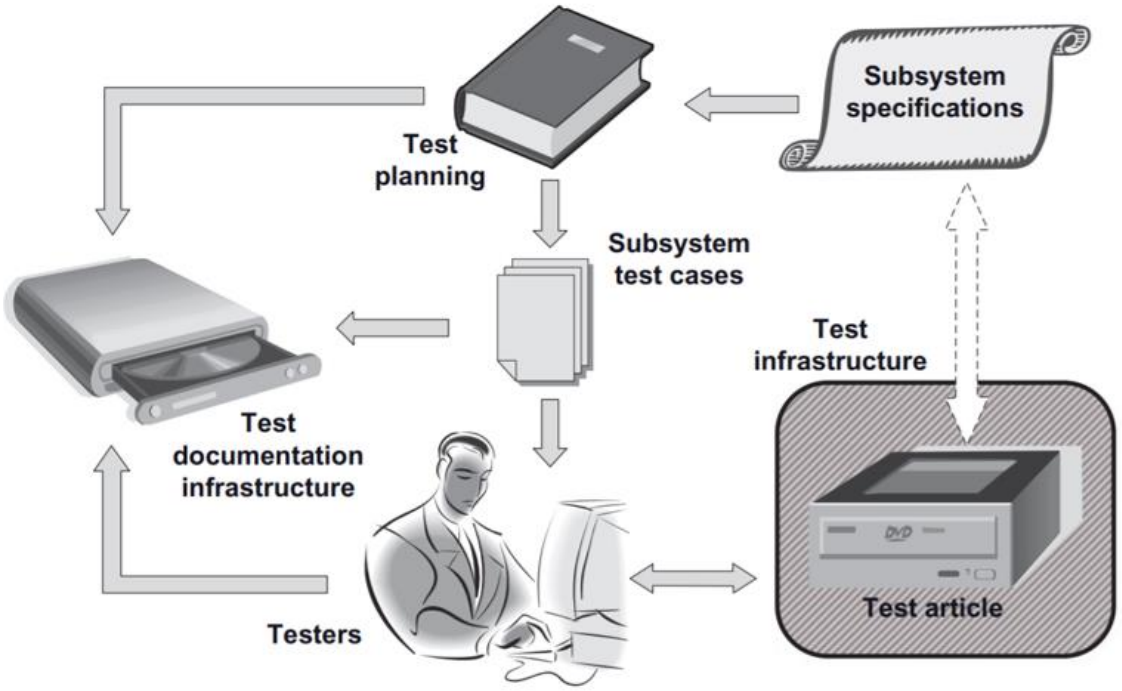


Figure 3. Preparing the test cycle for subsystems and components

The test - planning document should define a specific policy regarding the level of testing required of products developed by subcontractors as well as Commercial Off The Shelf (COTS) products. A rather soft policy will mandate only a review of the testing documents produced by subcontractors and probably accepting COTS products without any functional testing.

VERIFICATION, VALIDATION, AND TESTING OF ENGINEERED SYSTEMS - AVNER ENGEL Pg 114.

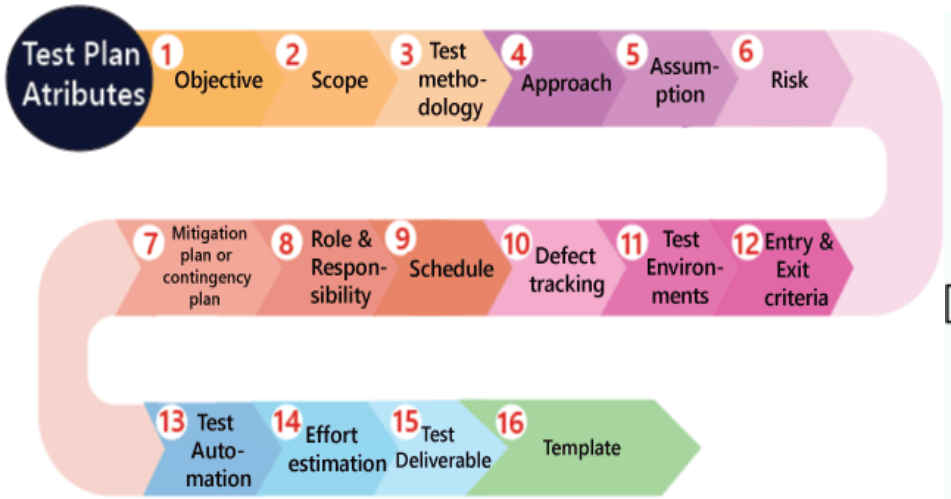
i. Software testing implementation - testing process - test plan, *(according to IEEE recommendations)*

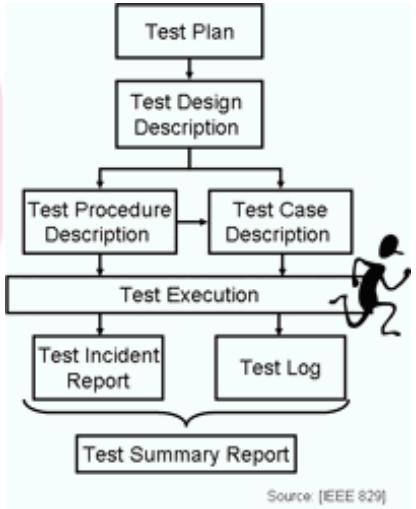
Test Plan

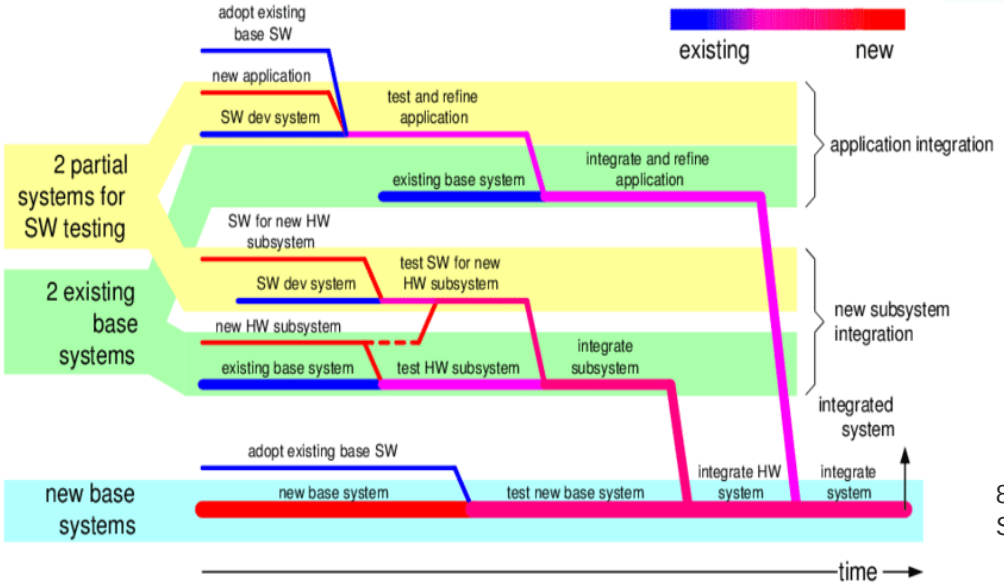
A test plan outlines the strategy that will be used to test an application, the resources that will be used, the test environment in which testing will be performed, and the limitations of the testing and the schedule of testing activities. Typically. the Quality Assurance Team Lead will be responsible for writing a Test Plan.

A test plan includes the following:

- ❑ Introduction to the Test Plan document
- ❑ Assumptions while testing the application
- ❑ List of test cases included in testing the application
- ❑ List of features to be tested
- ❑ What sort of approach to use while testing the software
- ❑ List of deliverables that need to be tested
- ❑ The resources allocated for testing the application
- ❑ Any risks involved during the testing process
- ❑ A schedule of tasks and milestones to be achieved







829-1983 - IEEE Standard for Software Test Documentation

VERIFICATION, VALIDATION, AND TESTING OF ENGINEERED SYSTEMS - AVNER ENGEL Pg 114.

i. Software testing implementation - Testing process - test case design Software and test report (STR) - *(according to IEEE recommendations)*

Test case design becomes more complicated as integration of the OO system begins. It is at this stage that testing of collaborations between classes must begin in Figure 4.

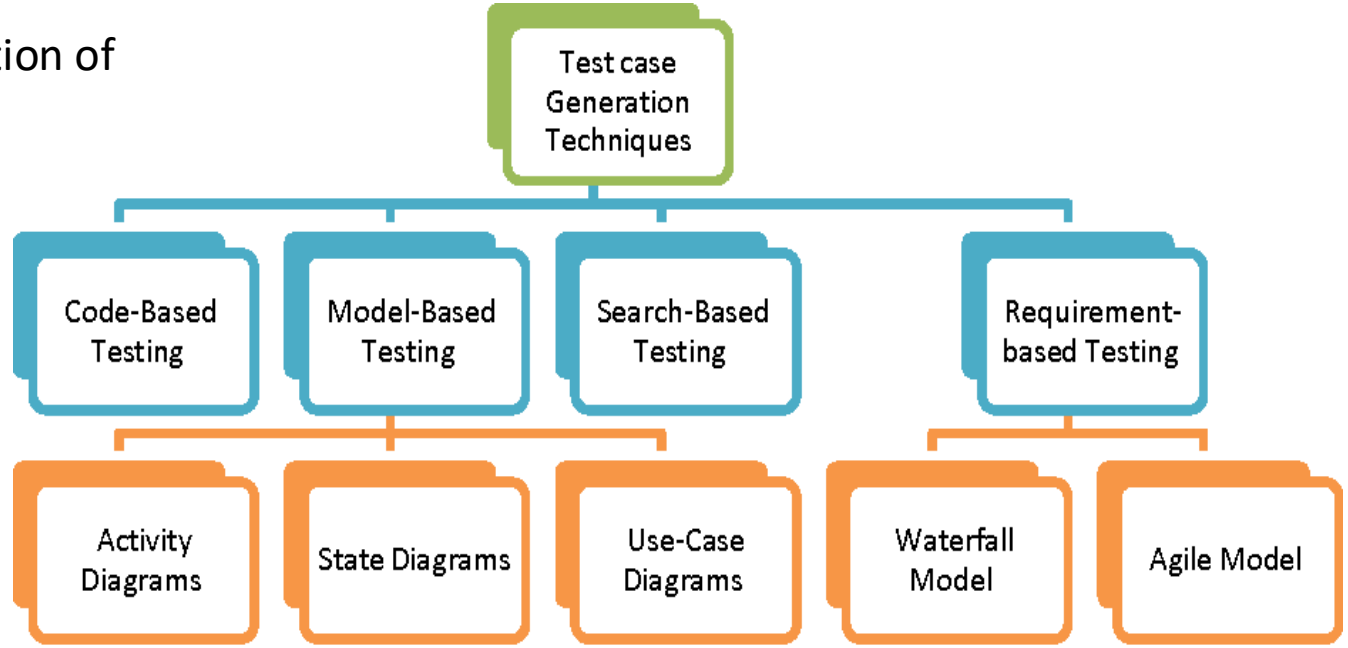


Figure 4. Survey Architecture

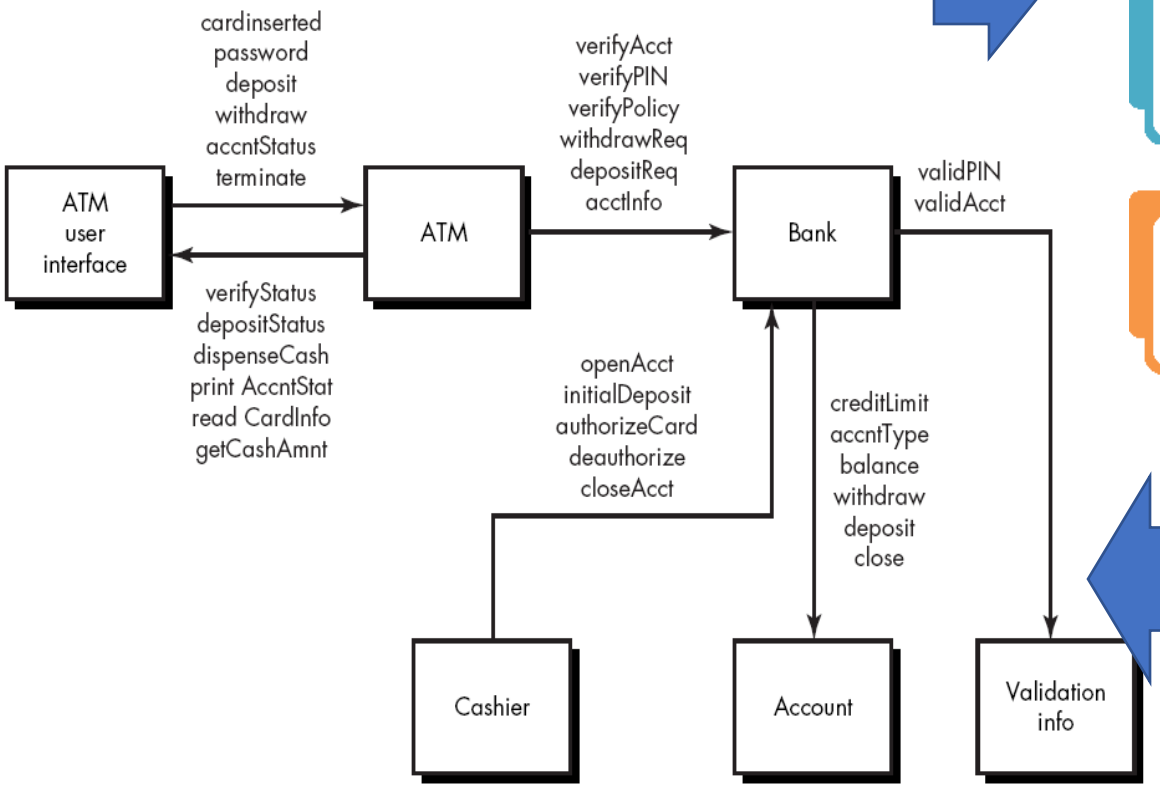


Figure 5. Banking Test Case Design

To illustrate "interclass test case generation" , I expand the banking example to include the classes and collaborations noted in Figure 5. The direction of the arrows in the Figure 5 indicates the direction of messages and the labeling indicates the operations that are invoked as a consequence of the collaborations implied by the messages.

i. Software testing implementation - Testing process - Software test report (STR) - *(according to IEEE recommendations)*

STR (Software/System Testing Results/Report) - what is it and how to use it?

STR = Software/System Testing Results/Report (all versions are in use)

Why do we need a STR?

STR gives a snapshot of a particular aspect of the **quality of the product** at the time of release - the state of tests run and results.

Assures that there will not be a situation in which test cases will not be performed and then forgotten.

STR is a **backward record**, if one day we want to know what the status of the tests of one version or another.

STR comes out with a version and in some industries, it must be **kept for a few years**.

Some companies separate the specific test results from the results summary report. Although there is a difference, the term STR includes both because there are no appropriate terminology for each, and they depend on each other.

Responsibility: Test manager / tester at the Agile group.

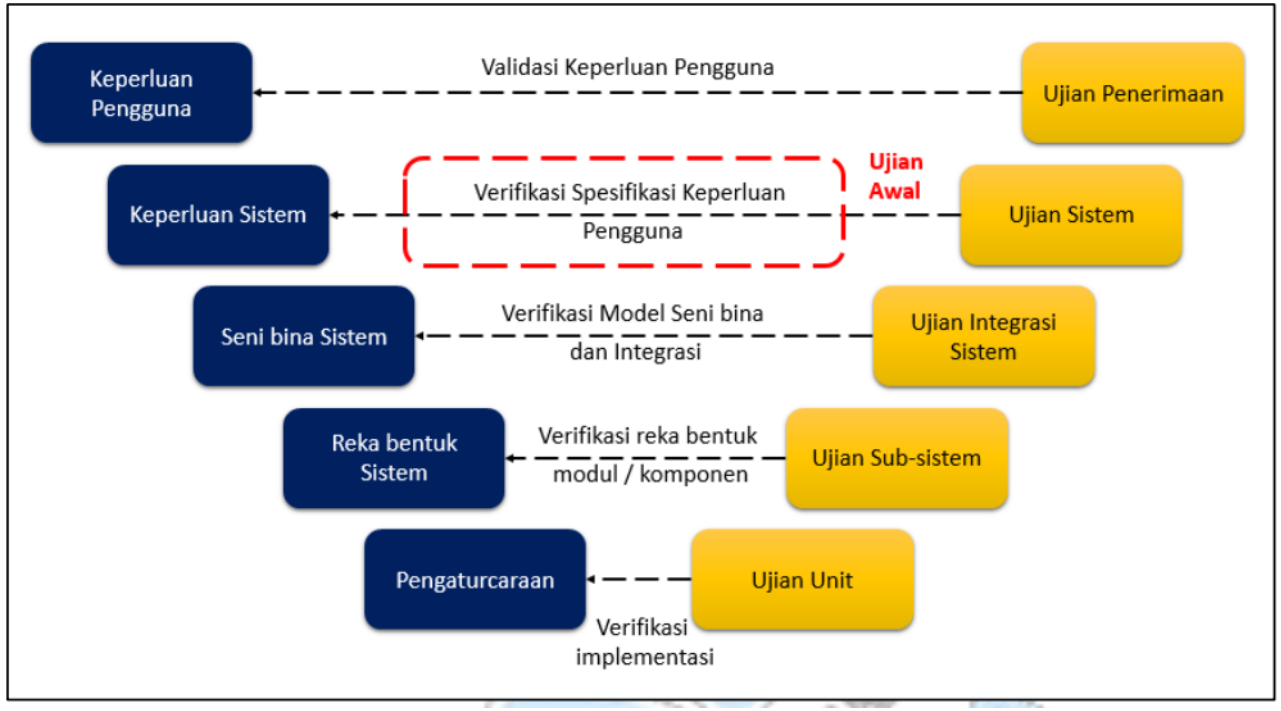
What is STR?

Once it was a physical document, and perhaps even now in certain industries like the pharmaceutical industry. Today the STR is viewed and managed in a test management tool such as **TestRail**. Beyond the obvious reasons (nature preservation, comfort, aesthetics) the advantage of these tools is the possibility of displaying the test summaries through graphs. In fact, in many cases you go over the totals - rather than specific tests - at the end of the version while deciding whether to release the product or not.

STR includes the results of the tests themselves and a summary of a look at the graphs.

STR is not the only consideration when discussing whether the version is good enough for release to the customer.

Software testing implementation – Types of V&V Testing and Description



Jenis Ujian	Keterangan
Pengujian Awal ***(Early Testing)***	Jenis pengujian adalah Ujian Statik dalam fasa permulaan projek. Pendekatan ujian adalah secara *review* atau/dan *walkthrough* ke atas keperluan fungsian (*functional*) dan bukan fungsian (*non-functional)* seperti yang dinyatakan dalam SRS. Artifak yang terlibat adalah seperti: • Dokumen tender • Cadangan tender daripada pembekal yang dilantik. • *Software Requirement Specification (SRS)*.
Ujian Unit (***Unit Testing***)	Aktiviti pengujian bagi menilai unit terkecil (lowest level) di dalam sistem seperti kod program, *function* dan sebagainya
Ujian Integrasi (***Integration Testing***)	Aktiviti pengujian dengan sistem-sistem dalaman dan sistem-sistem luaran yang terlibat secara terus dengan aplikasi yang sedang dibangunkan.
Ujian Sistem (***System Testing***)	Aktiviti pengujian yang terlibat adalah Pengujian Functional dan Non-Functional untuk keseluruhan modul yang dibangunkan
Ujian Penerimaan (***Acceptance Testing***)	Ujian Penerimaan Pengguna terbahagi kepada tiga (3) seperti turutan berikut: • Ujian Penerimaan Pengguna (UAT) • Ujian Penerimaan Sementara (PAT) • Ujian Penerimaan Akhir (FAT)

Source: KRISA Methodology, Malaysian Administrative Modernisation and Management Planning Unit (MAMPU)

WHY DO WE NEED UAT?

Software testing implementation – Types of user testing

- Alpha testing
  - Users of the software work with the development team to test the software at the developer's site.
- Beta testing
  - A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- Acceptance testing
  - Customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment. Primarily for custom systems.

Software testing implementation – User Acceptance Test Report

Project Name	Practicum Unit
Module/Sub-Module	AC10 Course Verification System for Intern
Testing Start Date / Time	30/01/23
Testing End Date / Time	30/01/23
Testing Justification	1. To test the CRUD function of the user 2. To test the CRUD function of the admin 3. To test the function of prerequisite 4. To test the search record/subject function
Name of Tester	Dr. Nor Asiah binti Mohamad@Razak

Test #	Steps to Execute	Expected Results	Pass	Fail	Defect/Comments/Additions
Application 1: Register					
1	Input the link to the website of the system	The login page of the website will be shown			
2	Click on the 'Register' link	The system will display the register page	/		
3	Input your name, matric number, password, batch number and click 'Register' button	System will be directed to the Main page (Subject Menu)	/		
Application 2: Login					
1	Input the link to the website of the system	The login page of the website will be shown			
2	Fill in the username and password and click Login'	System will display the Main page (Subject Menu) of the system	/		Please show the pop-up info for password security
Application 3: Add Record and View Record					
1	Click on the 'Add Subject' button	System will display the detail information need to be fulfilled	/		
2	Click 'Submit' after finish input all the details	System will display "Data created successfully"	/		
Application 4: Edit Record					
1	Click on the 'Edit' button	System will display the details of the subject	/		
2	Click 'Save Changes' after made the changes	System will display "Data updated successfully"	/		
Application 5: Delete Record					
1	Click 'Delete' button	System will display "Data deleted successfully"	/		
Application 6: Search Record					
1	Insert the code or the name of the subject and click 'Search'	System will display the searched subject	/		
Application 7: View and Download Prerequisite					
1	Click on the 'Prerequisite' button	System will display the summary of the prerequisite	/		
2	Click the download button	System will display the details of the qualification	/		
Application 8: Add Subject and View Subject					
1	Click 'Add Subject'	System will display the details need to be fulfilled	/		
2	Click 'Save'	System will display "Data created successfully"	/		
Application 9: Update Record					
1	Click 'Edit' button	System will display the details of the subject	/		
2	Click 'Save' after made changes	System will display "Data updated successfully"	/		
Application 10: Delete Record					
1	Click 'Delete' button	System will display "Data deleted successfully"	/		

Accepted	
Accepted with amendments	/
Not Accepted	

Tested by,

(DR. NOR ASIAH BINTI MOHAMAD@RAZAK)

WHY DO WE NEED UEQ?

Software testing implementation – UEQ: Instrument for data collection

What User Experience Questionnaire (UEQ) Measures?

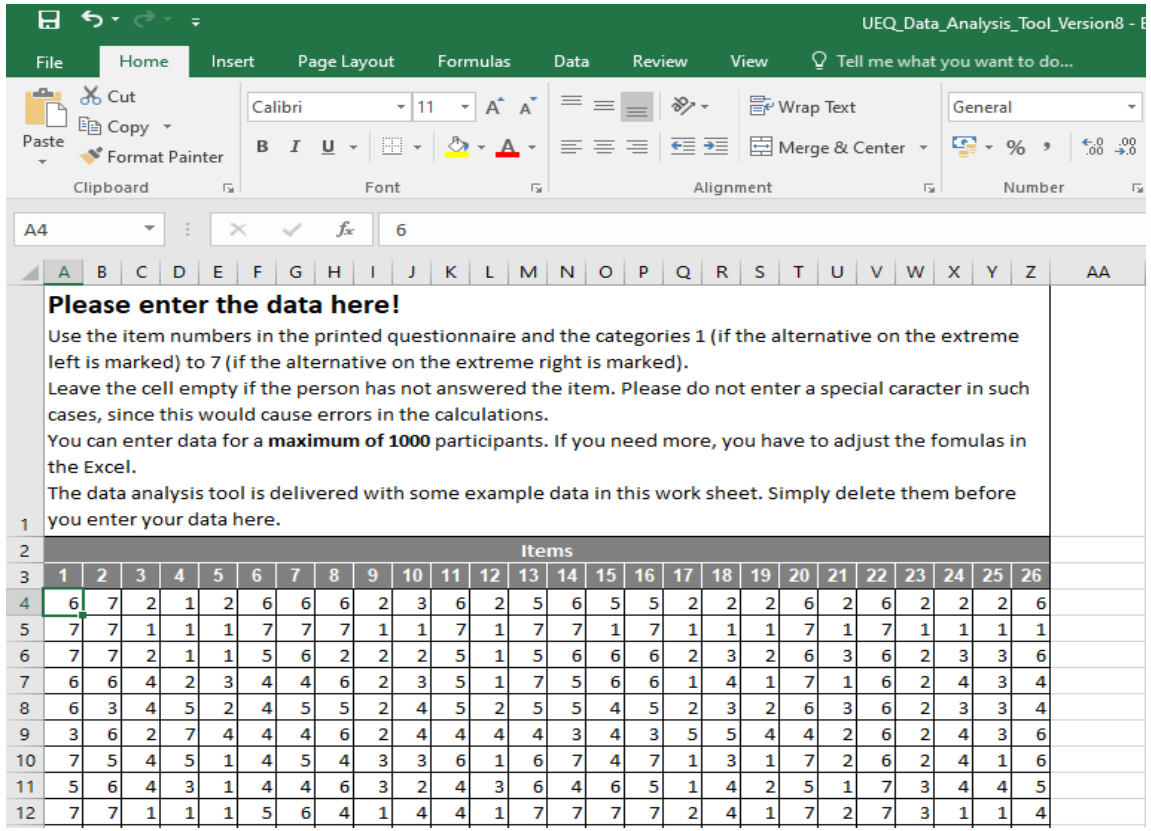
Attractiveness	• **Overall impression** of the product. Do users like or dislike it?
Perspicuity	• Is it **easy to get familiar** with the product and to learn how to use it?
Efficiency	• Can users solve their tasks without **unnecessary effort**?
Dependability	• Does the user feel **in control** of the interaction?
Stimulation	• Is it **exciting** and motivating to use the product? Is it **fun** to use?
Novelty	• Is the design of the product **creative**? Does it catch the interest of users?

User Experience Questionnaire (UEQ): Fast & Easy Way to Measure User Experience

Link to UEQ Material:
https://www.ueq-online.org/

Software testing implementation – UEQ: Tool for data analysis

Entering data

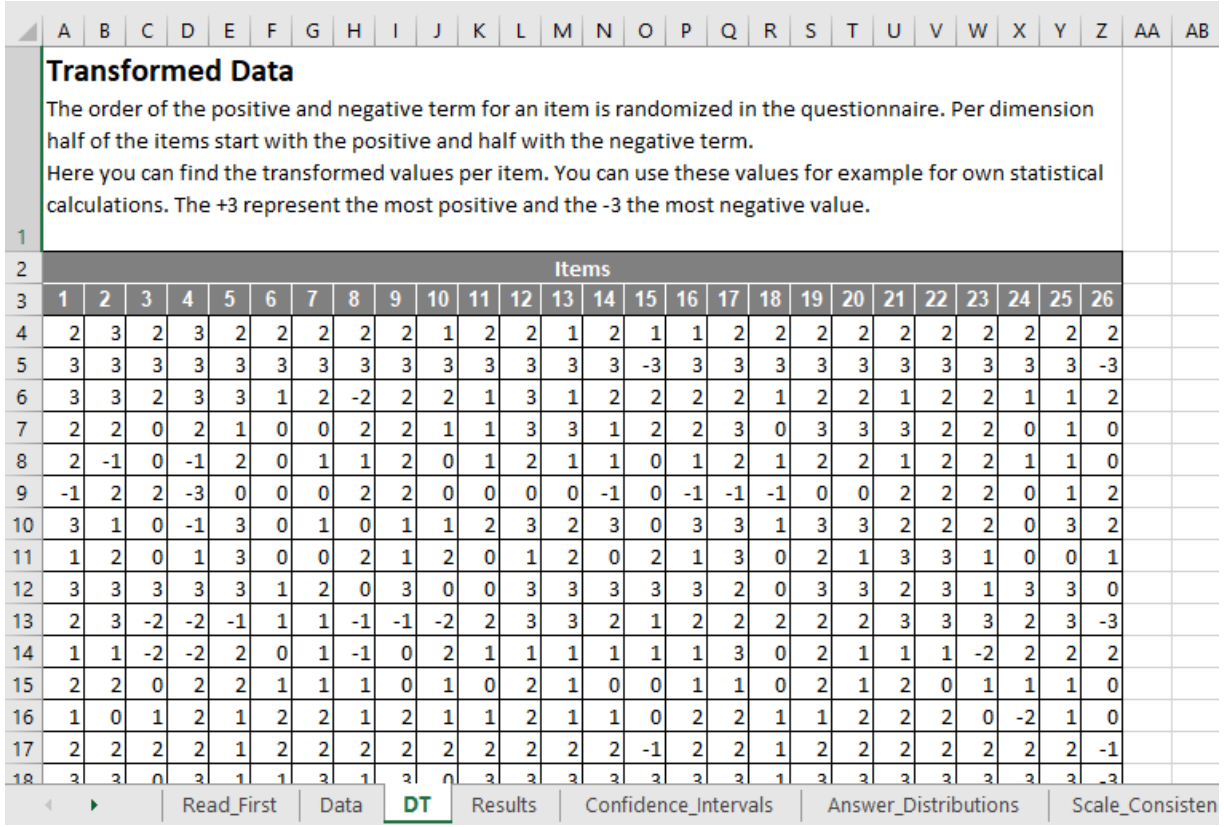


Please enter the data here!

Use the item numbers in the printed questionnaire and the categories 1 (if the alternative on the extreme left is marked) to 7 (if the alternative on the extreme right is marked).
Leave the cell empty if the person has not answered the item. Please do not enter a special caracter in such cases, since this would cause errors in the calculations.
You can enter data for a **maximum of 1000** participants. If you need more, you have to adjust the fomulas in the Excel.
The data analysis tool is delivered with some example data in this work sheet. Simply delete them before you enter your data here.

	Items																									
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
4	6	7	2	1	2	6	6	6	2	3	6	2	5	6	5	5	2	2	2	6	2	6	2	2	2	6
5	7	7	1	1	1	7	7	7	1	1	7	1	7	7	1	7	1	1	1	7	1	7	1	1	1	1
6	7	7	2	1	1	5	6	2	2	2	5	1	5	6	6	6	2	3	2	6	3	6	2	3	3	6
7	6	6	4	2	3	4	4	6	2	3	5	1	7	5	6	6	1	4	1	7	1	6	2	4	3	4
8	6	3	4	5	2	4	5	5	2	4	5	2	5	5	4	5	2	3	2	6	3	6	2	3	3	4
9	3	6	2	7	4	4	4	6	2	4	4	4	4	3	4	3	5	5	4	4	2	6	2	4	3	6
10	7	5	4	5	1	4	5	4	3	3	6	1	6	7	4	7	1	3	1	7	2	6	2	4	1	6
11	5	6	4	3	1	4	4	6	3	2	4	3	6	4	6	5	1	4	2	5	1	7	3	4	4	5
12	7	7	1	1	1	5	6	4	1	4	4	1	7	7	7	7	2	4	1	7	2	7	3	1	1	4

Transformed data



Transformed Data

The order of the positive and negative term for an item is randomized in the questionnaire. Per dimension half of the items start with the positive and half with the negative term.
Here you can find the transformed values per item. You can use these values for example for own statistical calculations. The +3 represent the most positive and the -3 the most negative value.

	Items																									
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
4	2	3	2	3	2	2	2	2	2	1	2	2	1	2	1	1	2	2	2	2	2	2	2	2	2	2
5	3	3	3	3	3	3	3	3	3	3	3	3	3	3	-3	3	3	3	3	3	3	3	3	3	3	-3
6	3	3	2	3	3	1	2	-2	2	2	1	3	1	2	2	2	2	1	2	2	1	2	2	1	1	2
7	2	2	0	2	1	0	0	2	2	1	1	3	3	1	2	2	3	0	3	3	3	2	2	0	1	0
8	2	-1	0	-1	2	0	1	1	2	0	1	2	1	1	0	1	2	1	2	2	1	2	2	1	1	0
9	-1	2	2	-3	0	0	0	2	2	0	0	0	0	-1	0	-1	-1	-1	0	0	2	2	2	0	1	2
10	3	1	0	-1	3	0	1	0	1	1	2	3	2	3	0	3	3	1	3	3	2	2	2	0	3	2
11	1	2	0	1	3	0	0	2	1	2	0	1	2	0	2	1	3	0	2	1	3	3	1	0	0	1
12	3	3	3	3	3	1	2	0	3	0	0	3	3	3	3	3	2	0	3	3	2	3	1	3	3	0
13	2	3	-2	-2	-1	1	1	-1	-1	-2	2	3	3	2	1	2	2	2	2	2	3	3	3	2	3	-3
14	1	1	-2	-2	2	0	1	-1	0	2	1	1	1	1	1	1	3	0	2	1	1	1	-2	2	2	2
15	2	2	0	2	2	1	1	1	0	1	0	2	1	0	0	1	1	0	2	1	2	0	1	1	1	0
16	1	0	1	2	1	2	2	1	2	1	1	2	1	1	0	2	2	1	1	2	2	2	0	-2	1	0
17	2	2	2	2	1	2	2	2	2	2	2	2	2	2	-1	2	2	1	2	2	2	2	2	2	2	-1
18	3	3	0	3	1	1	3	1	3	0	3	3	3	3	3	3	3	1	3	3	3	3	3	3	3	-3

Read_First | Data | DT | Results | Confidence_Intervals | Answer_Distributions | Scale_Consisten

Software testing implementation – UEQ: Tool for data analysis

Mean score PER participant

AB	AC	AD	AE	AF	AG	AH
	Scale means per person					
	Attractiveness	Perspicuity	Efficiency	Dependability	Stimulation	Novelty
	1.83	2.25	2.00	2.00	2.00	1.50
	3.00	3.00	3.00	3.00	3.00	0.00
	2.00	2.00	2.00	0.75	1.75	2.00
	1.50	2.50	2.25	2.25	0.25	0.75
	1.33	0.00	2.00	1.50	1.00	0.00
	-0.33	0.25	1.50	0.25	-0.25	1.00
	2.50	1.00	2.00	2.00	1.25	0.75
	0.50	2.00	1.50	1.75	0.75	1.25
	3.00	2.75	2.50	1.25	1.50	1.50
	2.33	1.75	1.75	1.25	0.75	-1.50
	1.33	0.25	0.00	1.25	0.75	0.75
	1.17	1.75	0.50	1.00	1.00	0.25
	0.83	1.25	1.50	1.25	1.50	0.50
	2.00	2.00	2.00	2.00	1.50	0.50
	3.00	3.00	3.00	2.50	1.50	0.00
	2.83	2.75	2.00	2.00	2.50	2.50
	-1.83	1.00	1.50	1.00	-1.25	0.50
	2.33	1.75	2.00	0.50	0.75	0.50
	2.33	2.50	2.50	1.75	1.25	1.50
	2.00	1.75	2.25	2.00	1.75	2.75
	2.00	1.50	2.00	1.50	1.75	2.00

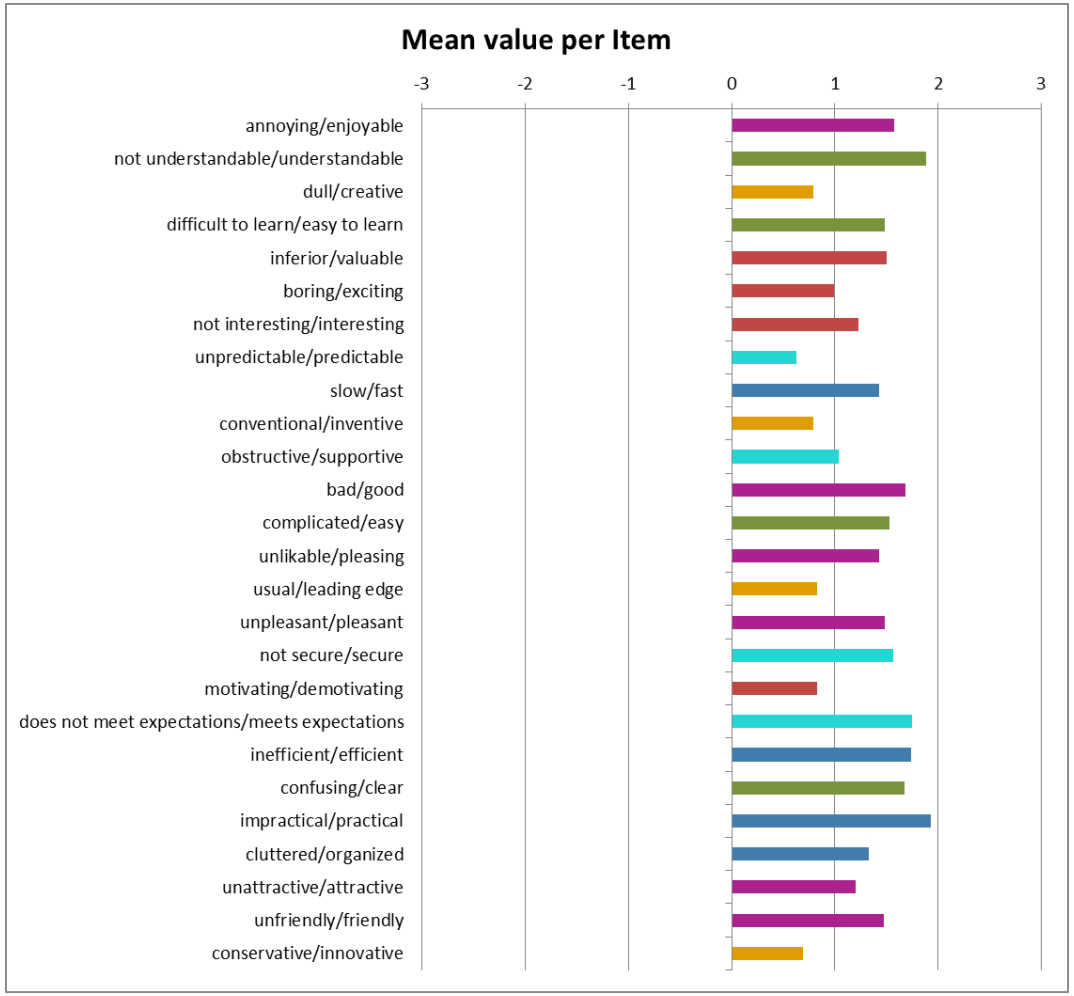
Scale_Consistency | Benchmark | Inconsistencies | Sample_Size | KPI_Ca ...

Result – Mean score PER item

	A	B	C	D	E	F	G	H	I
2	Item	Mean	Variance	Std. Dev.	No.	Left	Right	Scale	
3	1	1.6	1.9	1.4	255	annoying	enjoyable	Attractiveness	
4	2	1.9	1.7	1.3	255	not understandable	understandable	Perspicuity	
5	3	0.8	2.3	1.5	255	creative	dull	Novelty	
6	4	1.5	2.7	1.6	255	easy to learn	difficult to learn	Perspicuity	
7	5	1.5	2.2	1.5	255	valuable	inferior	Stimulation	
8	6	1.0	1.7	1.3	255	boring	exciting	Stimulation	
9	7	1.2	1.9	1.4	255	not interesting	interesting	Stimulation	
10	8	0.6	1.8	1.3	255	unpredictable	predictable	Dependability	
11	9	1.4	1.8	1.3	255	fast	slow	Efficiency	
12	10	0.8	2.2	1.5	255	inventive	conventional	Novelty	
13	11	1.0	1.8	1.3	255	obstructive	supportive	Dependability	
14	12	1.7	2.1	1.4	255	good	bad	Attractiveness	
15	13	1.5	2.0	1.4	255	complicated	easy	Perspicuity	
16	14	1.4	2.1	1.4	255	unlikable	pleasing	Attractiveness	
17	15	0.8	2.1	1.5	255	usual	leading edge	Novelty	
18	16	1.5	1.9	1.4	255	unpleasant	pleasant	Attractiveness	
19	17	1.6	2.1	1.5	255	secure	not secure	Dependability	
20	18	0.8	1.7	1.3	255	motivating	demotivating	Stimulation	
21	19	1.7	1.9	1.4	255	meets expectations	does not meet expectations	Dependability	
22	20	1.7	1.8	1.3	255	inefficient	efficient	Efficiency	
23	21	1.7	1.9	1.4	255	clear	confusing	Perspicuity	
24	22	1.9	1.2	1.1	255	impractical	practical	Efficiency	
25	23	1.3	2.6	1.6	255	organized	cluttered	Efficiency	
26	24	1.2	1.8	1.3	255	attractive	unattractive	Attractiveness	
27	25	1.5	1.8	1.3	255	friendly	unfriendly	Attractiveness	
28	26	0.7	2.2	1.5	255	conservative	innovative	Novelty	

Software testing implementation – UEQ: Tool for data analysis

Graph – Mean score PER item



Result – Mean score PER scale

UEQ Scales (Mean and Variance)		
Attractiveness	1.473	1.31
Perspicuity	1.642	1.29
Efficiency	1.607	1.10
Dependability	1.243	0.83
Stimulation	1.135	1.13
Novelty	0.774	1.17

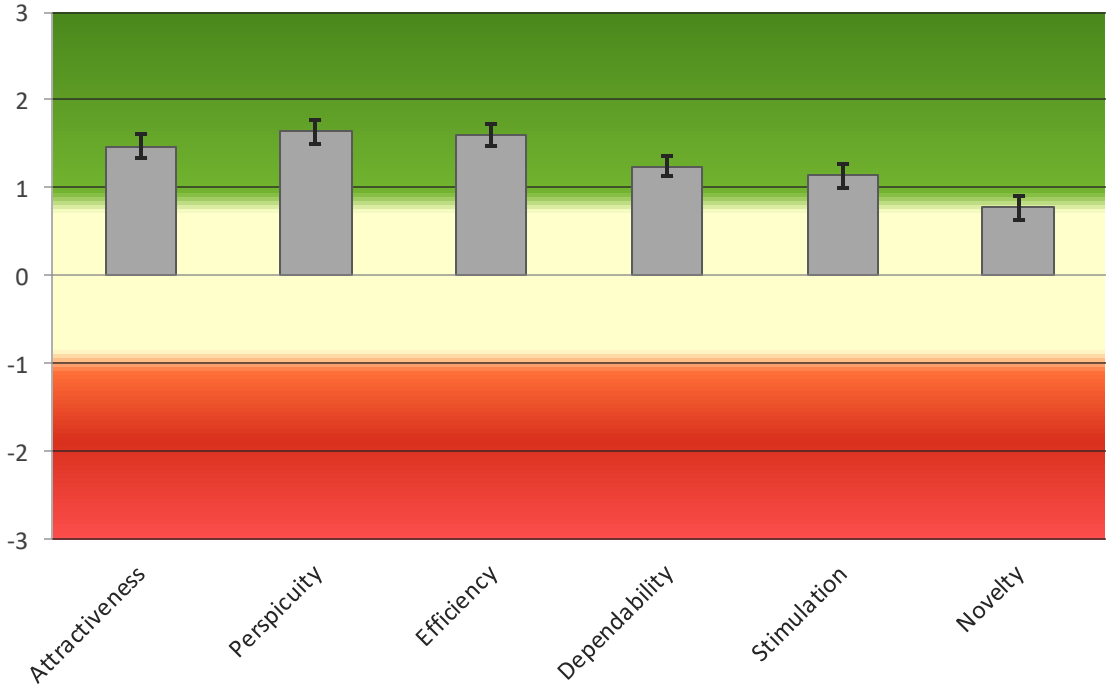
Software testing implementation – UEQ: Tool for data analysis

Result – Confidence interval

Confidence intervals (p=0.05) per scale						
Scale	Mean	Std. Dev.	N	Confidence	Confidence interval	
Attractiveness	1.473	1.143	255	0.140	1.333	1.613
Perspicuity	1.642	1.138	255	0.140	1.502	1.782
Efficiency	1.607	1.048	255	0.129	1.478	1.735
Dependability	1.243	0.908	255	0.111	1.132	1.355
Stimulation	1.135	1.061	255	0.130	1.005	1.266
Novelty	0.774	1.079	255	0.132	0.641	0.906

Confidence intervals (p=0.05) per scale

Graph – Mean score PER scale



Software testing implementation – UEQ: Tool for data analysis

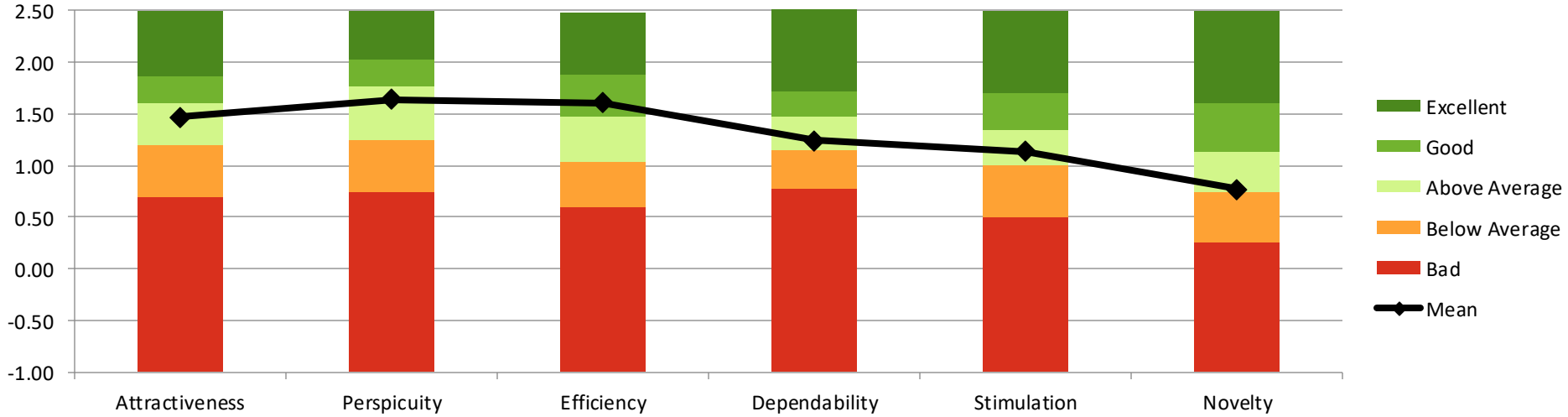
Benchmarking

Scale	Mean	Comparisson to benchmark	Interpretation
Attractiveness	1.47	Above average	25% of results better, 50% of results worse
Perspicuity	1.64	Above Average	25% of results better, 50% of results worse
Efficiency	1.61	Good	10% of results better, 75% of results worse
Dependability	1.24	Above Average	25% of results better, 50% of results worse
Stimulation	1.14	Above Average	25% of results better, 50% of results worse
Novelty	0.77	Above Average	25% of results better, 50% of results worse

The scale means are set in relation to existing benchmark data set (20190 persons from 452 studies)

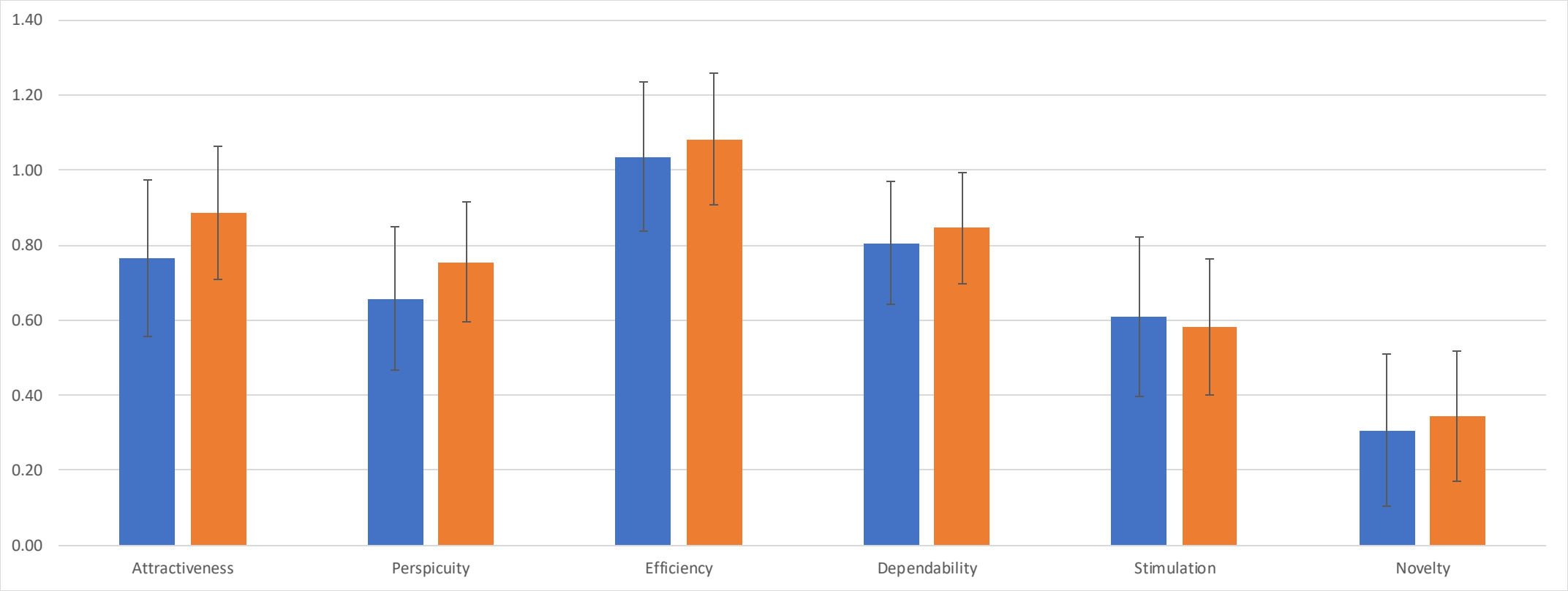
Benchmark allows conclusions about the relative quality of the evaluated product compared to other products.

Benchmarking graph



Software testing implementation – UEQ: Tool for data analysis

Comparing MEAN SCORE for version 1 & version 2



END

UNIVERSITI PENDIDIKAN SULTAN IDRIS
اونيۏرسيتي ڤنديديقن سلطان ادريس
SULTAN IDRIS EDUCATION UNIVERSITY

Thank You…
Terima kasih..